EXPT NO: 1	A PYTHON PROGRAM TO SOLVE THE TRANSPORTATION PROBLEM
DATE:	

AIM:

To solve the Transportation Problem for minimizing the cost of transporting goods from multiple sources to multiple destinations using Python.

PROCEDURE:

1. INSTALL REQUIRED LIBRARIES:

    • Make sure you have numpy and scipy installed. You can install them using pip if not already installed.

    pip install numpy scipy

2. DEFINE THE PROBLEM:

    • Identify the cost matrix, supply array, and demand array.

3. SET UP THE PROBLEM IN PYTHON:

    • Use the scipy.optimize.linprog function to set up and solve the Transportation Problem.

4. RUN THE CODE:

    • Execute the Python code to get the optimized result.

5. ANALYZE THE OUTPUT:

    • Interpret the results and validate the solution against the problem constraints.

PROBLEM 1: SOLVE THE IFBS FOR THE PROBLEM

	D1	D2	D3	SUPPLY
	4	8	8	50
	2	7	6	40
	3	4	2	60
DEMAND	30	70	50	

CODE

```
import numpy as np

from scipy.optimize import linprog

def transportation_problem(supply, demand, costs):

supply = supply.copy() demand

= demand.copy()

if sum(supply) != sum(demand):

if sum(supply) < sum(demand): diff =

sum(demand) - sum(supply)

supply.append(diff)

costs = np.hstack((costs, np.zeros((costs.shape[0], 1))))

else:

diff = sum(supply) - sum(demand) demand.append(diff)

costs = np.vstack((costs, np.zeros((1, costs.shape[1]))))

num_sources = len(supply) num_destinations

= len(demand)

A_eq = np.zeros((num_sources + num_destinations, num_sources *

num_destinations))

for i in range(num_sources):

A_eq[i, i * num_destinations:(i + 1) * num_destinations] = 1

for j in range(num_destinations):

A_eq[num_sources + j, j::num_destinations] = 1

b_eq = np.array(supply + demand) c =

costs.flatten()
```

```
result = linprog(c, A_eq=A_eq, b_eq=b_eq, method='highs')

if result.success:

allocation = result.x.reshape(num_sources, num_destinations)

total_cost = result.fun return allocation, total_cost

else:

raise Exception("Linear programming failed: " + result.message)

supply= [50, 40, 60] demand =

[30, 70, 50]

costs = np.array([[4, 2, 3],

[8, 7, 4],

[8, 6, 2]])

allocation, total_cost = transportation_problem(supply, demand, costs)

print("Optimal Allocation:") print(allocation)

print("Total Cost:", total_cost)
```

OUTPUT

```
Optimal Allocation:

[[ 0. 50. 0.]

[30. 10. 0.]

[ 0. 10. 50.]]

Total Cost: 570.0
```

PROGRAM 2: SOLVE THE IFBS FOR THE PROBLEM

	D1	D2	D3	SUPPLY
	5	3	2	10
	4	2	8	20
	4	6	3	30
DEMAND	15	25	20	60

CODE

```
import numpy as np from scipy.optimize import linprog
```

```
def transportation_problem(supply, demand, costs): supply = supply.copy() demand =
demand.copy()

# Balance the problem
if sum(supply) != sum(demand):
    if sum(supply) < sum(demand):
        diff = sum(demand) - sum(supply)
        supply.append(diff)
        costs = np.hstack((costs, np.zeros((costs.shape[0], 1))))
    else:
        diff = sum(supply) - sum(demand)
        demand.append(diff)
        costs = np.vstack((costs, np.zeros((1, costs.shape[1]))))

num_sources = len(supply)
num_destinations = len(demand)

# Create constraint matrix
A_eq = np.zeros((num_sources + num_destinations,
                num_sources * num_destinations))

for i in range(num_sources):
    A_eq[i, i * num_destinations:(i + 1) * num_destinations] = 1

for j in range(num_destinations):
    A_eq[num_sources + j, j::num_destinations] = 1

b_eq = np.array(supply + demand)

# Cost function
c = costs.flatten()

# Solve using linear programming
result = linprog(c, A_eq=A_eq, b_eq=b_eq, method='highs')

if result.success:
    allocation = result.x.reshape(num_sources, num_destinations)
    total_cost = result.fun
    return allocation, total_cost
else:
    raise Exception("Linear programming failed: " + result.message)


supply = [10,20,30] demand = [15, 25 20]
```

```
costs = np.array([ [5,4,4], [3,2,6], [2,8,3] ])

allocation, total_cost = transportation_problem(supply, demand, cost)

print("Optimal Allocation:") print(allocation)

print("Total Cost:", total_cost)
```

OUTPUT

```
Optimal Allocation:

 [[ 0 5 5]

[0 20 0]

[ 15 0 15]]

Total Cost: 155
```

RESULT

Thus The Transportation Problem For Minimizing The Cost Of Transporting Goods From Multiple Sources To Multiple Destinations Using Python Was Executed Successfully.